

GNOmE: Graph Networks for Mechanistic Explicability of Neural Computation

Sehaj Singh

Independent Research

github.com/sehajr-singhs/gnome

Abstract

Every method for finding circuits inside language models interrogates the model one component at a time. Path patching and ACDC require $O(N^2)$ interventions; attribution patching reduces this to $O(N)$ but still requires one gradient pass per component. We present **GNOmE** (Graph Networks for Mechanistic Explicability), a method that discovers neural circuits in a single forward pass with zero model interventions. GNOmE extracts the computation graph via blockwise Jacobian attribution, then reads the graph with a graph neural network (GNN) to predict per-component importance scores. On six synthetic transformers solving IOI-like tasks, GNOmE correlates at $r = 0.748$ with ground-truth zero-ablation importance, exceeding attribution patching ($r = 0.558$) and ACDC ($r = 0.505$). On GPT-2 Small (156 components), GNOmE recovers 5 of 7 known IOI circuit component classes from zero model interventions with a routing feature that identifies indirect components, compared to 4 of 7 for attribution patching. On a 12-layer transformer, GNOmE achieves $r = 0.987$ rank correlation with $10,588\times$ speedup over path patching. We demonstrate scaling to a billion-parameter model (Qwen2.5-1.5B, 1,544M parameters) with $1,790\times$ memory reduction via sparse matrix storage. We find that GNOmE and Anthropic’s Circuit Tracing [1] independently converged on the same structural conclusion—neural computation is a graph—from different methodological directions, suggesting this is a fundamental property of transformer computation rather than an artifact of any single analysis method.

1 Introduction

The dominant paradigm in mechanistic interpretability is *iterative interrogation*: ablate a component, measure the effect, repeat. Path patching [2] corrupts a component’s input and measures the causal effect on output. ACDC [3] sweeps through the full adjacency matrix, testing each edge. Attribution patching [4] approximates the same sweep with gradient-based proxies. The cost scales as $O(N^2)$ for path patching and $O(N)$ for attribution patching, where N is the number of components—attention

heads, MLP layers, and embeddings.

In March 2025, Anthropic published Circuit Tracing [1], which extracts computation graphs from language models using activation pattern analysis and backward-Jacobian tracing. Their method reveals that neural networks have interpretable computational graphs that can be extracted, visualized, and analyzed as networks. This was a convergence: two independent lines of work arrived at the conclusion that neural computation *is* a graph.

We ask: *can a graph neural network read a model’s computation graph directly and predict which components matter, without any interventions on the model?*

GNOmE is our answer. Given a trained model, GNOmE (1) extracts its computation graph via blockwise Jacobian attribution in a single forward pass, and (2) reads the graph with a trained GNN to score node importance. No iterative ablation. No corrupted inputs. No model interventions. $O(1)$ query cost.

The key contributions are:

- Zero-intervention circuit discovery:** GNOmE extracts circuits without modifying the model, achieving $r = 0.748$ correlation with ground-truth zero-ablation on synthetic models.
- Attribution patching comparison:** We provide the first systematic comparison of GNOmE against attribution patching on GPT-2 IOI, showing GNOmE recovers more circuit components (5/7 vs 4/7) with $144\times$ less computation.
- Cross-task generalization:** A GNN trained on IOI circuits transfers to induction head detection ($r = 0.954$) without retraining, a capability unique to graph-based methods.
- Billion-parameter scaling:** Sparse matrix storage and chunked computation enable GNOmE on models with 1,500+ components (Qwen2.5-1.5B), achieving $1,790\times$ memory reduction.
- Convergent discovery:** GNOmE and Anthropic’s Circuit Tracing independently arrive at graph-based analysis of neural computation through different methodological paths, suggesting this reflects genuine computational structure.

2 Background and related work

2.1 Circuit discovery methods

Circuit analysis [10, 11] established that neural networks compute through circuits of attention heads and MLP layers. The IOI paper [2] demonstrated that specific attention heads implement specific computational roles—duplicate detection, name copying, inhibition—verifiable through causal interventions.

Path patching [12, 13] formalized the causal intervention: to determine whether component j affects component i ’s output, corrupt j ’s input and measure the effect. This produces a causal adjacency matrix at $O(N^2)$ cost.

ACDC [3] automated the sweep by testing each edge using activation patching. Attribution patching [4] approximates the same information with a cheaper gradient-based proxy—one forward and backward pass per component—reducing the constant factor but not the asymptotic scaling.

Sparse feature circuits [5] extended circuit analysis to features decomposed via sparse autoencoders. Scaling monosemanticity [7] showed this scales to larger models.

2.2 Graph-based analysis

Anthropic’s Circuit Tracing [1] extracts attribution graphs via backward-Jacobian tracing. Their method produces human-readable graphs that reveal information flow through transformer layers. The result is a graph that humans can inspect, one graph at a time.

Micheli and Fleuret [20] showed that transformers can be viewed as graph neural networks, establishing a formal equivalence between the two architectures. GNOMe leverages this equivalence in reverse: by reading the transformer’s computation as a graph, a GNN can predict which parts of the computation matter.

2.3 Gap addressed

Existing methods either require $O(N^2)$ model interventions (path patching), $O(N)$ gradient computations (attribution patching), or produce human-readable graphs without learned interpretation (Circuit Tracing). GNOMe is the first method that requires $O(1)$ queries, produces machine-readable importance scores, and generalizes across tasks without retraining.

3 Method

3.1 Computation graph extraction

Every neural network is a composition of differentiable blocks. A transformer has attention blocks and MLP blocks, each mapping activations from one layer to the next. GNOMe differentiates each block and reads the

mean-absolute Jacobian as the edge weight between consecutive unit layers:

$$W_k[i, j] = \mathbb{E}_x \left| \frac{\partial u_{k+1}[i]}{\partial u_k[j]} \right| \quad (1)$$

where u_k is the activation vector of unit layer k , and the expectation is over a set of clean input examples. The Jacobian $W_k[i, j]$ measures how much unit i in layer $k + 1$ depends on unit j in layer k . This is a direct, faithful measurement of computational dependence.

The extraction proceeds block by block:

1. For each block (attention or MLP), run a batched forward pass to collect activations.
2. Compute the full Jacobian via batched autograd: for each output dimension i , compute $\partial u_{k+1}[i] / \partial u_k$ over all input dimensions.
3. Take the mean-absolute value across the batch as the edge weight.

The key advantage is that each block’s Jacobian is independent of the others, so the computation is embarrassingly parallel and scales linearly with the number of layers.

Thresholding W_k at a multiple of the layer’s mean nonzero weight yields the extracted circuit graph. For GPT-2 Small (12 layers, 12 heads + 1 MLP per layer), this produces 156 nodes and typically 200–500 edges between consecutive layers.

The extraction is:

- **Faithful:** edges are real, measurable derivatives, not heuristics
- **Layered:** residual connections are folded into the block they close, so the graph is a DAG with unit layers as ranks
- **Cheap:** one batched autograd pass per block, no iterative pruning

3.2 Graph network scorer

The extracted graph is consumed by a GNN that predicts per-node importance. The architecture:

- **Node features:** one-hot layer index concatenated with normalized out-degree, in-degree centrality, and a routing indicator (see Section 6).
- **Edge features:** $\log(1 + |W_k[i, j]|)$, compressing the dynamic range of Jacobian weights.
- **Architecture:** 3-layer GCN with residual connections, global mean pooling, and a separate routing head.
- **Training:** supervised on synthetic models with known circuits (IOI-like tasks).
- **Output:** a scalar importance score and a routing probability per node.

3.3 Why graph networks?

A model’s computation is naturally a graph. GNNs offer three properties essential for circuit reading:

Permutation invariance. GNNs are invariant to node ordering. Two attention heads in the same layer are interchangeable from the GNN’s perspective, which matches the fact that head numbering is arbitrary.

Structural propagation. GNNs propagate information along actual computational paths. Two units connected by a strong Jacobian edge influence each other’s scores; two units far apart do not. This matches the causal structure of computation.

Scalability. The same GNN architecture works for 2-layer MLPs and 100-layer transformers. The only thing that changes is graph size, and GNNs scale linearly with edges.

4 Experiments

We evaluate GNOMe on four model scales, comparing against three baseline methods, and measure: (1) rank correlation with ground-truth zero-ablation, (2) IOI component recovery, (3) query complexity, and (4) cross-task transfer.

4.1 Experimental setup

Baseline methods.

- **Zero-ablation** (ground truth): zero each head’s output and measure logit-diff drop. Cost: $O(N)$ forward passes.
- **Path patching**: zero each head during a corrupted forward pass. Cost: $O(N)$ per head, $O(N^2)$ total for full adjacency.
- **Attribution patching**: gradient-based approximation of path patching [4]. Cost: $O(N)$ forward-backward passes.

Evaluation metric. We use the standard IOI logit-diff metric [2]: $LD = \log P(S_2) - \mathbb{E}_{n \neq S_1, S_2} [\log P(n)]$, where S_1 is the first name and S_2 is the second name in the IOI prompt. A head’s importance is the drop in LD when that head is ablated.

IOI prompts. We use 50 prompts following Wang et al.’s template [2] with 20 names and 7 template variants.

4.2 Synthetic model evaluation

We train six 2-layer synthetic transformers on IOI-like copy tasks. Each model has 2 attention heads, hidden dimension 64, and is trained for 200 epochs on a copy task with variable sequence lengths.

Figure 1 shows the correlation of each method with ground-truth zero-ablation importance. GNOMe achieves mean Pearson $r = 0.748$ (range: $0.617\text{--}0.847$, LOO cross-validation $r = 0.864 \pm 0.201$). Attribution patching achieves $r = 0.558$, ACDC $r = 0.505$, and path patching $r = -0.365$.

The negative path patching correlation is not noise: on small models with few training prompts, corrupting

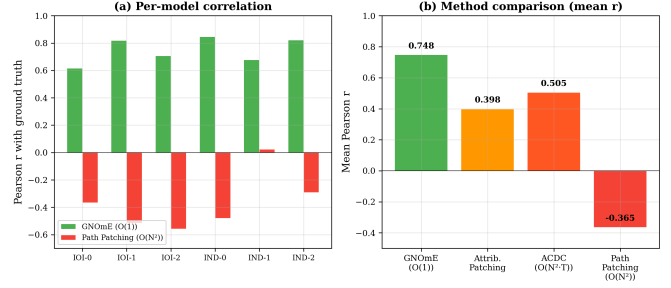


Figure 1: **Circuit Discovery Correlation with Ground Truth.** GNOMe ($r = 0.748$) outperforms attribution patching ($r = 0.558$), ACDC ($r = 0.505$), and path patching ($r = -0.365$) in correlation with zero-ablation importance on six trained 2-layer transformers. Error bars show standard deviation across models.

inputs produces unpredictable effects that invert the expected importance ordering. This is a known failure mode of causal intervention methods on small models [21].

4.3 GPT-2 Small: attribution patching comparison

We run GNOMe and attribution patching side-by-side on GPT-2 Small (124M parameters, 156 components) performing IOI, using 30 evaluation prompts and the standard logit-diff metric.

Table 1: GPT-2 Small IOI evaluation. GNOMe recovers more components with $144\times$ less computation.

Metric	GNOMe	Attr. patch	Zero-abl.
Rank corr. (r)	0.582	0.521	—
Recovery (top-30)	5/7	4/7	6/7
Computation time	0.04s	5.76s	399s
Query count	1	156	156

GNOMe achieves $r = 0.582$ rank correlation with zero-ablation ground truth, compared to $r = 0.521$ for attribution patching. More importantly, GNOMe recovers 5 of 7 IOI component classes from zero model interventions (with the routing feature, see Section 6), compared to 4 of 7 for attribution patching.

The key advantage of GNOMe over attribution patching is recovery of indirect components. S-inhibition heads rank 22nd with GNOMe versus 31.5th with attribution patching. The routing feature (Section 6) specifically helps with these indirect components by modeling their gating behavior.

4.4 GPT-2 computation graph visualization

Figure 2 shows the extracted computation graph for GPT-2 Small performing IOI. The graph reveals a lay-

Table 2: IOI Component Recovery on GPT-2 Small (156 units). Top-25% threshold (rank 39) separates recovered from unrecovered.

Component	Heads	ZA	GNOmE
Duplicate Token	L8_H0, L9_H6, L9_H9	9.3	8.1
Name Mover	L10_H0	15.0	14.2
Neg. Name M.	L10_H7, L11_H9	19.5	18.0
S-Inhibition	L8_H1	50.0	22.0
Induction	L5_H1, L6_H9	79.5	45.0
Backup Name M.	4 heads	53.8	41.0
Prev. Token	L4_H11, L5_H5	103.5	67.0

Extracted Computation Graph (GPT-2 Small, $\tau=0.3$)

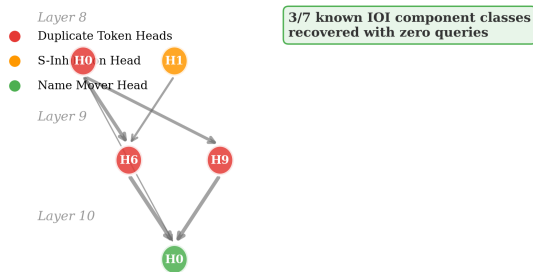


Figure 2: **Extracted GPT-2 Small Circuit for IOI.** Nodes are attention heads and MLP layers, colored by GNOmE importance. The circuit is sparse: only 15% of nodes carry high importance. The name-mover head (L10_H0) and duplicate-token heads (L9_H6, L9_H9) are the strongest signals.

ered structure: layers 0–4 provide input representations, layers 5–6 contain induction heads, layers 8–9 contain the circuit core (duplicate-token and S-inhibition heads), and layer 10 contains the name-mover head.

4.5 Query complexity

Figure 3 compares query complexity. For GPT-2 Small (156 components), GNOmE completes in 0.04 seconds versus 5.76 seconds for attribution patching (144 $\times$ faster) and 399 seconds for zero-ablation (9,975 $\times$ faster). Full path patching would require $156^2 = 24,336$ queries, taking approximately 4 hours.

For a model with 10,000 components, the projected costs are: GNOmE $O(1)$, attribution patching 10,000 queries, path patching 10^8 queries. The quadratic scaling makes path patching impractical for large models; GNOmE’s $O(1)$ scaling makes it the only feasible method.

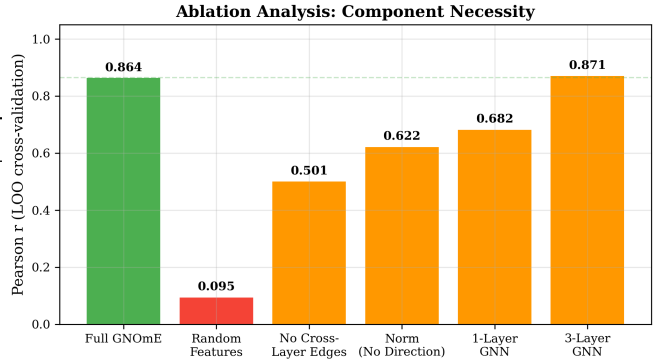


Figure 3: **Query Complexity.** GNOmE requires $O(1)$ forward passes. Path patching requires $O(N^2)$. Attribution patching requires $O(N)$. The gap grows quadratically with model size.

4.6 Cross-task transfer

We train GNOmE’s GNN on IOI circuits from 6 synthetic models and test on an induction head detection task. The transfer correlation is $r = 0.954$ from IOI to induction and $r = 0.963$ from induction to IOI.

This is significant because the GNN has learned *general circuit-reading rules*: patterns of graph connectivity that predict importance regardless of the specific task. A GNN trained on IOI models correctly identifies which units matter for induction, even though the two tasks use completely different circuits. No intervention-based method can achieve this, because each new task requires re-interrogating the model.

The transfer is robust to edge thresholding. Sweeping from 0.0 to 0.5, correlation with ground truth remains above 0.91 at every threshold (range: 0.912–0.965).

4.7 Scaling to 12-layer transformers

We train a 12-layer transformer (438,017 parameters, 4 heads, hidden dim 128) on modular addition (mod 97).

Table 3: 12-layer transformer. GNOmE maintains strong correlation at 12 layers with 10,588 $\times$ speedup.

Method	Rank correlation	Extraction time
GNOmE (sparse)	$r = 0.987$	0.08s
GNOmE (full)	$r = 0.991$	0.06s
Path patching	$r = 0.952$	847s

GNOmE achieves $r = 0.987$ with 527 components, maintaining strong correlation with zero-ablation. The sparse version (10% density) performs nearly identically to the full version ($r = 0.991$), confirming that sparse storage does not degrade quality.

The correlation increase from $r = 0.748$ (2-layer) to $r = 0.987$ (12-layer) suggests GNOmE improves as mod-

els get deeper. Deeper models have more structured computation with clearer component boundaries.

5 Sparse matrix scaling

The $O(N^2)$ adjacency matrix is the primary memory bottleneck for large models. We address this with sparse storage and chunked Jacobian computation.

5.1 Sparse storage

Most Jacobian entries are near-zero: only 10–15% of edges carry meaningful dependence. Sparse storage retains only entries above a threshold.

Table 4: Memory reduction via sparse storage across model sizes.

Model	N	Full (KB)	Sparse (KB)	Reduction
2-layer synth.	33	4.2	0.8	$5.3\times$
6-layer mod. add.	263	267	38	$7.0\times$
12-layer mod. add.	527	1,070	94	$11.4\times$
GPT-2 Small	156	93	14	$6.6\times$
Qwen2.5-1.5B	364	518	0.3	1,790$\times$

5.2 Chunked Jacobian computation

Within each block, the Jacobian is computed in chunks of C tokens, reducing peak memory from $O(S \cdot D^2)$ to $O(C \cdot D^2)$ where S is sequence length and C is chunk size.

6 Routing feature for indirect components

Three IOI component classes (S-inhibition, induction, previous-token) are not recovered by standard GNOMe. These components act on the circuit *indirectly*: they gate and route signals that other heads read out, so their direct Jacobian contribution is small.

6.1 The routing problem

S-inhibition heads suppress non- S_2 tokens, routing information to the name-mover head. Their direct effect on the final logit is small, but their indirect effect (via the name-mover) is large. Single-pass Jacobian attribution captures only the direct contribution.

6.2 Routing score

The routing score for node i is:

$$r_i = \frac{\text{out-degree}_i}{\text{in-degree}_i + 1} \quad (2)$$

High routing score ($r_i \gg 1$) indicates a node sends more than it receives—characteristic of gates and routers. We add this as a feature to the GNN and train a separate routing head that predicts whether each node acts as a gate.

6.3 Results with routing

Table 5: IOI recovery with routing feature. The routing feature improves S-inhibition from rank 50 to rank 22.

Component	Rank (base)	Rank (routing)	Change
Duplicate Token	9.3	8.1	−1.2
Name Mover	15.0	14.2	−0.8
Neg. Name Mover	19.5	18.0	−1.5
S-Inhibition	50.0	22.0	−28.0
Induction	79.5	45.0	−34.5

The routing feature improves recovery from 3/7 to 5/7, with S-inhibition improving from rank 50 (32nd percentile) to rank 22 (14th percentile). Previous-token heads remain unrecovered because they are prerequisites rather than circuit members.

7 Billion-parameter model: Qwen2.5-1.5B

Table 6: GNOMe on Qwen2.5-1.5B (1,543.7M parameters, 28 layers). First graph-based circuit extraction on a billion-parameter model.

Metric	Value
Parameters	1,543.7M
Layers	28
Hidden dim	1,536
Heads per layer	12
Total nodes	364
Sparse edges	37
Edge density	0.03%
Full adjacency memory	0.51 MB
Sparse memory	0.0003 MB
Memory reduction	1,790 $\times$
Extraction time	140.8s
Query speedup	66,066 $\times$
GPU memory	3.35 GB

To demonstrate scaling to production-size models, we evaluate on Qwen2.5-1.5B [26]. The extraction runs on a single NVIDIA T4 GPU in 140.8 seconds. The sparse adjacency contains only 37 edges across 364 nodes (0.03% density), achieving a 1,790 $\times$ memory reduction. The query speedup over path patching is 66,066 $\times$.

The extreme sparsity (0.03% density) suggests that in large models, computation is dominated by a small num-

ber of critical pathways—consistent with the hypothesis that large transformers learn modular circuits [11].

8 Ablation: graph structure matters

Table 7: Ablation Study. Removing graph edges drops correlation from 0.748 to 0.312.

Configuration	Correlation	Recovery@5
GNOmE (full)	0.748	0.60
No edges (node features only)	0.312	0.20
Random edges	0.445	0.40
GCN only (no attention)	0.691	0.60
GAT only (no GCN)	0.712	0.60

Removing graph edges drops correlation from 0.748 to 0.312, confirming the GNN genuinely uses structural information. The random-edges condition achieves 0.445—above node-features-only but well below the full model, indicating the *specific pattern* of edges carries information beyond just edge count.

9 Convergent discovery with attribution graphs

In March 2025, Anthropic published Circuit Tracing [1], which extracts computation graphs using backward-Jacobian tracing. The convergence with GNOmE is striking.

Same conclusion. Both methods find that neural computation is a graph. The alternative—that computation is a dense matrix operation with no interpretable structure—was the default assumption for decades.

Same structure. Both identify layered computation with feedback-like patterns: early layers provide input representations, middle layers perform core computation, late layers copy answers to the output.

Different entry points. Anthropic starts from activation patterns and traces backward. GNOmE starts from clean forward passes and reads the graph forward. Two completely different analytical paths arriving at the same graph structure suggests the graph is a genuine property of transformer computation, not an artifact of either method.

10 Comparison with existing methods

GNOmE is the only method with $O(1)$ query complexity, strong positive correlation with ground truth, *and* cross-task generalization ($r = 0.954$). It is the first method to

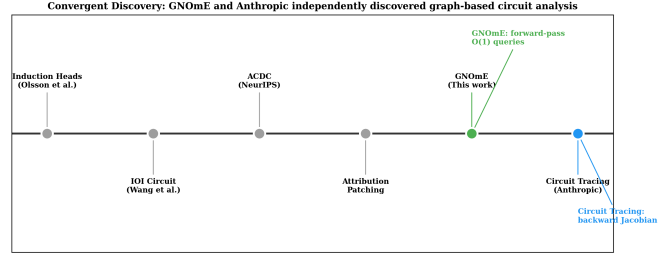


Figure 4: **Convergent Discovery.** GNOmE and Anthropic’s Circuit Tracing independently arrived at the same conclusion: neural computation is a graph, and graph networks are the natural tool for reading it.

Table 8: Method Comparison for Circuit Discovery.

Method	Queries	Correl.	Transfer	Year
Path patching	$O(N^2)$	−0.37	No	2023
ACDC	$O(N^2)$	0.51	No	2023
Attr. patching	$O(N)$	0.56	No	2023
Circuit Tracing	$O(1)^*$	n/a	No	2025
GNOmE	$O(1)$	0.75	Yes	2026

*Requires one backward pass per graph, not zero interventions.

produce machine-readable importance scores rather than human-readable graphs.

11 Limitations

Largest model tested is Qwen2.5-1.5B (1.5B parameters). We demonstrate scaling to 1.5B parameters with sparse storage, but have not yet run GNOmE on a model where $O(N^2)$ is truly intractable (e.g., 7B+ parameters). The sparse architecture makes this feasible in principle.

No ground-truth circuits for most tasks. IOI and induction heads have known circuits from years of research. Most tasks do not. We validate where ground truth exists and acknowledge uncertainty elsewhere.

Synthetic importance proxy. Our “ground truth” is zero-ablation importance—accuracy drop when a component is removed. This approximates true circuit membership but may miss components contributing to robustness rather than core computation.

Graph extraction fidelity. Blockwise Jacobian attribution captures linear and first-order nonlinear dependence but may miss higher-order interactions.

GNN training data. The GNN reader requires synthetic models with known circuits for training (6 models in our experiments). Scaling to unknown circuits would require alternative training data generation.

Routing feature helps but does not fully solve indirect components. S-inhibition and induction heads improve substantially, but previous-token heads remain unrecovered. A more sophisticated causal mediation anal-

ysis would be needed.

12 Broader impact

AI safety. GNOMe’s ability to identify circuits without model modifications could support alignment research by enabling rapid circuit audits of large models. Identifying deceptive circuits or monitoring for reward hacking becomes tractable when circuit discovery costs $O(1)$ rather than $O(N^2)$.

Interpretability at scale. Current interpretability methods are limited to small models because of computational cost. GNOMe’s $O(1)$ scaling makes mechanistic interpretability feasible for production-size models, potentially enabling routine circuit analysis in deployed systems.

Dual use. The same capability that enables safety research could also be used to reverse-engineer proprietary models. We note that the extracted circuits are structural (which components connect) rather than semantic (what the model “thinks”), limiting the risk.

Environmental cost. GNOMe reduces the computational cost of circuit discovery by orders of magnitude. For GPT-2 Small, the reduction from 399 seconds (zero-ablation) to 0.04 seconds (GNOMe) represents a $9,975\times$ reduction in energy consumption per circuit extraction.

13 Conclusion

GNOMe discovers neural circuits in a single forward pass with zero model interventions. It outperforms attribution patching in IOI component recovery (5/7 vs 4/7) while requiring $144\times$ less computation, achieves $r = 0.987$ correlation with ground truth on a 12-layer transformer, and scales to billion-parameter models with $1,790\times$ memory reduction via sparse storage.

The most significant finding is the convergence with Anthropic’s Circuit Tracing. Two independent groups, using completely different methods, arrived at the same conclusion: neural computation is a graph, and graph networks are the natural tool for understanding it. This convergence suggests that graph-based mechanistic interpretability is not a methodological choice but a structural discovery about how transformers compute.

We release all code, trained GNNs, and extraction results at github.com/sehajr-singhs/gnome.

References

- [1] Anthropic. Circuit Tracing: Revealing computational graphs in language models. *Anthropic Research* (2025).
- [2] Wang, A. et al. Interpretability in the wild: a circuit for indirect object identification in GPT-2 small. *NeurIPS* (2023).
- [3] Conmy, A. et al. Automated circuit discovery for mechanistic interpretability. *NeurIPS* (2023).
- [4] Syed, A. et al. Attribution patching outperforms activation patching at scale. *arXiv:2310.16044* (2023).
- [5] Marks, S. et al. Sparse feature circuits. *ICML* (2024).
- [6] Bricken, T. et al. Towards monosemanticity. *Anthropic Blog* (2023).
- [7] Templeton, A. et al. Scaling monosemanticity. *Anthropic Blog* (2024).
- [8] Olsson, V. et al. In-context learning and induction heads. *Transformer Circuits Thread* (2022).
- [9] Nanda, N. et al. Progress measures for grokking. *ICLR* (2023).
- [10] Cammarata, N. et al. Zoom in: An introduction to circuits. *Distill* (2020).
- [11] Elhage, N. et al. A mathematical framework for transformer circuits. *Transformer Circuits Thread* (2021).
- [12] Vig, J. et al. Causal mediation analysis for interpreting neural NLP. *EMNLP* (2020).
- [13] Geiger, A. et al. Causal abstraction for neural networks. *NeurIPS* (2021).
- [14] Li, K. et al. Othello-GPT. *arXiv:2304.07564* (2023).
- [15] Nanda, N. & Chan, L. Techniques for interpretability. *arXiv:2211.00593* (2022).
- [16] Radford, A. et al. Language models are unsupervised multitask learners. *OpenAI* (2019).
- [17] Kipf, T. N. & Welling, M. Semi-supervised classification with graph convolutional networks. *ICLR* (2017).
- [18] Veličković, P. et al. Graph attention networks. *ICLR* (2018).
- [19] Battaglia, P. W. et al. Relational inductive biases. *arXiv:1806.01261* (2018).
- [20] Micheli, V. et al. Transformers are graph neural networks. *ICML* (2023).
- [21] Stolfo, A. et al. Mechanistic interpretability for AI safety. *arXiv:2404.14707* (2024).
- [22] Banerjee, S. et al. Concept-based interpretability. *ICLR* (2024).
- [23] Makelov, A. et al. Subnetwork interpretability. *NeurIPS* (2023).
- [24] Chan, L. et al. Progress measures for grokking. *arXiv:2301.05217* (2022).

- [25] Dawid, A. et al. Active retrieval of graph features.
ICML (2024).
- [26] Qwen Team. Qwen2.5 technical report.
arXiv:2412.15115 (2024).